

Oyente vs. Oyente+: Evaluating a Vulnerability Scanner Update

Thomas Fenninger, Technische Universität Wien

July 23, 2025

Abstract

Oyente¹ could not analyze contracts written for the latest EVM and Solidity releases. We therefore forked it into Oyente+², adding full support for current EVM and Solidity versions.

To validate these enhancements, we conducted a comparative evaluation using subsets of these two smart contract datasets: cgt³ and skelcodes⁴. We integrated Oyente+ as a new tool into SmartBugs⁵—generating normalized outputs (CSV or PostgreSQL arrays) of both tools and making analysis of the result data easier.

We assessed performance, stability and the new features of Oyente+ by executing multiple iterations on each dataset—analyzing runtime bytecodes and Solidity source files. We tested for regressions introduced by Oyente+. Detailed changes and enhancements in Oyente+ are documented in the repository⁶.

¹<https://github.com/smartbugs/oyente>

²https://github.com/smartbugs/oyente_plus

³<https://github.com/gsalzer/cgt>

⁴<https://github.com/gsalzer/skelcodes>

⁵<https://github.com/smartbugs/smartbugs>

⁶https://github.com/smartbugs/oyente_plus/blob/update/EVMandSOLupdate.md

Contents

1	Introduction And Motivation	4
2	Generating Results	4
2.1	SmartBugs Execution	4
2.2	Database Setup	4
3	Data Overview	4
3.1	Contract Dataset Statistics	4
3.2	Solidity Version Distribution	5
3.2.1	Further Analysis of the Skelcodes Dataset	5
3.3	Vulnerability Types	5
3.4	SmartBugs Schema	6
4	Run Summary	7
5	Results: Vulnerability Comparison	7
5.1	Callstack Depth Attack Vulnerability Analysis	8
5.2	Integer Overflow Analysis	8
5.3	Integer Underflow Analysis	9
5.4	Re-Entrancy Vulnerability Analysis	9
5.5	Timestamp Dependency Analysis	9
5.6	Transaction Ordering Dependence Analysis	10
6	Error And Failure Analysis	11
6.1	Error Analysis	11
6.1.1	Error Type Breakdown	11
6.2	Failure Analysis	12
6.2.1	Failure Type Details	12
7	Runtime Performance and Timeout Analysis	14
7.1	EVM Code Coverage Analysis	14
8	Areas of Improvement in Oyente+	16
9	Conclusion	17

List of Tables

1	Overview of contract source files in each dataset.	5
2	Distribution of Solidity versions across datasets.	5
3	Solidity release versions and their dates.	6
4	Summary of SmartBugs runs comparing Oyente and Oyente+.	7
5	Detection summary for <i>Callstack_Depth_Attack_Vulnerability</i>	8
6	Detection summary for <i>Integer_Overflow</i>	9
7	Detection summary for <i>Integer_Underflow</i>	9
8	Detection summary for <i>Re_Entrancy_Vulnerability</i>	10
9	Detection summary for <i>Timestamp_Dependency</i>	10
10	Detection summary for <i>Transaction_Ordering_Dependence_TOD</i>	10
11	Summary of contracts with errors detected by Oyente and Oyente+.	11
12	Most frequent errors encountered during analysis.	12
13	Contracts that ended in a “fail” status per run.	12
14	Detailed failure types by run, normalized to exception or Docker exit codes.	13
15	Summary of successful-run durations and timeout counts.	14
16	Average EVM code coverage by run and tool.	15

1 Introduction And Motivation

Smart contracts—self-executing programs deployed on blockchain platforms such as Ethereum—have ushered in a new era of decentralized applications. However, they remain prone to critical vulnerabilities (e.g. re-entrancy, integer over/underflows, timestamp dependence) that attackers can exploit for financial gain. Static analysis tools like Oyente perform symbolic execution to uncover these flaws, but the original Oyente is limited to EVM v1.7.3 and Solidity $\leq 0.4.26$. As the Ethereum ecosystem has evolved, this version gap has rendered vanilla Oyente unable to analyze the vast majority of recent contracts.

In this work, we present **Oyente+**, a fork of Oyente that:

- Fully Supports Solidity 0.5.x–0.8.x and EVM releases up to v1.16.1
- Integrates seamlessly into the SmartBugs framework for large-scale evaluation

2 Generating Results

This section explains how we produced the results presented in this report. It covers (1) the SmartBugs execution parameters we chose and (2) the database setup used to collect and analyze the tool outputs.

2.1 SmartBugs Execution

We installed SmartBugs following the official guide⁷ and then tailored its settings to our hardware. In particular, the flags `processes`, `timeout` and `mem-limit` should be set according to the target system to ensure efficient, reproducible analyses. A running Docker daemon is a prerequisite of SmartBugs—consider that it uses the python-lib `docker`, which relies on the Docker socket to communicate with the daemon. It is recommended to use Docker itself, instead of other tools (e.g., SUSE Rancher Desktop) to avoid issues. See Appendix A for the full installation and run commands.

2.2 Database Setup

To enable SQL-based analysis, we used a PostgreSQL database instead of importing SmartBugs’ normalized CSV output into a spreadsheet. On Ubuntu 24.04.2 LTS, we installed PostgreSQL, created a database, defined a table matching the CSV schema, and loaded the data via PostgreSQL `COPY` command. You can find the full list of database installation and setup commands in Appendix B.

3 Data Overview

This section introduces the two smart contract datasets we analyzed (1) and highlights their key characteristics relevant to this evaluation (2). Furthermore the section describes which vulnerabilities Oyente and Oyente+ are looking for (3) and how the normalized data of SmartBugs is structured (4).

To get a deeper understanding of the datasets, one can take a look at their respective GitHub repositories⁸⁹.

3.1 Contract Dataset Statistics

Table1 summarizes the composition of each dataset in terms of folder count, the number of files declaring a pragma, files without any pragma, and files containing multiple pragmas. Each folder contains a `<contract>.sol`, `<contract>.hex` and `<contract>.rt.hex` file of the respective smart contract.

This gives an immediate sense of how many contracts might produce errors during and require parsing or cleanup before analysis.

⁷<https://github.com/smartbugs/smartbugs/blob/master/doc/installation.md>

⁸<https://github.com/gsalzer/cgt>

⁹<https://github.com/gsalzer/skelcodes>

Dataset	Folders	Files w/ pragma	Files w/o pragma	Multi-pragma files
CGT	1879	1462	417	17
Skelcodes	1799	1795	4	315

Table 1: Overview of contract source files in each dataset.

3.2 Solidity Version Distribution

Table 2 shows how the two datasets break down by Solidity pragmas in the source code (0.4.x through 0.8.x). These are minimal versions the bytecode has to be compiled with, but do not reflect the actual solc versions used.

Version	CGT count	Skelcodes count
0.4.x	1454	1914
0.5.x	15	3522
0.6.x	49	3528
0.7.x	0	2235
0.8.x	0	3837

Table 2: Distribution of Solidity versions across datasets.

The CGT dataset has 64 contracts with a Solidity version not supported by Oyente.

3.2.1 Further Analysis of the Skelcodes Dataset

For the Skelcodes dataset we did some further analysis and parsed the file `info.csv`¹⁰, which contains the solc version it was compiled with. We extracted all lines of our sample and then took a list of unique solc versions. Furthermore their release dates were captured from GitHub¹¹. In Table 3 the results are shown.

3.3 Vulnerability Types

We focus on the six common vulnerability classes detected by Oyente and Oyente+:

Callstack Depth Attack Vulnerability: An attacker repeatedly chains external calls until the EVM’s maximum call-stack depth is reached, causing subsequent calls to fail and potentially masking malicious behavior.¹² This vulnerability has been made largely irrelevant by an EVM update that was deployed on the main net in 2016.

Integer Overflow: Occurs when an arithmetic operation exceeds the maximum value of its integer type (e.g. adding 1 to a `uint8(255)` wraps to 0), leading to incorrect logic or security breaches.¹³ Since version 0.8.0, the Solidity compiler adds bound checks by default, hence overflows will manifest as assertion violations.

Integer Underflow: The inverse of overflow: subtracting below the minimum of an integer wraps to its maximum (e.g., $0 - 1$ wraps to $2^n - 1$), which can be exploited to bypass checks or drain funds.¹⁴ Since version 0.8.0, the Solidity compiler adds bound checks by default, hence underflows will manifest as assertion violations.

¹⁰<https://github.com/gsalzer/skelcodes/blob/main/info.csv>

¹¹All release-tag URLs available on the official GitHub Releases page: <https://github.com/ethereum/solidity/releases/tag/v<version>>.

¹²<https://docs.soliditylang.org/en/latest/security-considerations.html>

¹³<https://owasp.org/www-project-smart-contract-top-10/2023/en/src/SC02-integer-overflow-underflow.html>

¹⁴<https://owasp.org/www-project-smart-contract-top-10/2023/en/src/SC02-integer-overflow-underflow.html>

Version	Release Date	Contract Count	Version	Release Date	Contract Count
0.5.0	2018-11-13	36	0.5.1	2018-12-03	36
0.5.2	2018-12-19	36	0.5.3	2019-01-22	36
0.5.4	2019-02-12	36	0.5.5	2019-03-05	36
0.5.6	2019-03-13	36	0.5.7	2019-03-26	36
0.5.8	2019-04-30	36	0.5.9	2019-05-28	36
0.5.10	2019-06-25	36	0.5.11	2019-08-12	36
0.5.12	2019-10-01	36	0.5.13	2019-11-14	36
0.5.14	2019-12-09	36	0.5.15	2019-12-17	36
0.5.16	2020-01-02	36	0.5.17	2020-03-17	36
0.6.0	2019-12-17	36	0.6.1	2020-01-02	36
0.6.2	2020-01-27	36	0.6.3	2020-02-18	36
0.6.4	2020-03-10	36	0.6.5	2020-04-06	36
0.6.6	2020-04-09	36	0.6.7	2020-05-04	35
0.6.8	2020-05-14	36	0.6.9	2020-06-05	36
0.6.10	2020-06-11	36	0.6.11	2020-07-07	36
0.6.12	2020-07-22	36	0.7.0	2020-07-28	36
0.7.1	2020-09-02	36	0.7.2	2020-09-28	36
0.7.3	2020-10-07	36	0.7.4	2020-10-19	36
0.7.5	2020-11-18	36	0.7.6	2020-12-16	36
0.8.0	2020-12-16	36	0.8.1	2021-01-27	36
0.8.2	2021-03-02	36	0.8.3	2021-03-23	36
0.8.4	2021-04-21	36	0.8.5	2021-06-10	36
0.8.6	2021-06-22	36	0.8.7	2021-08-11	36
0.8.8	2021-09-27	36	0.8.9	2021-09-29	36
0.8.10	2021-11-09	36	0.8.11	2021-12-20	36

Table 3: Solidity release versions and their dates.

Re-Entrancy Vulnerability: A contract makes an external call before updating its own state, allowing the callee to recursively re-enter and manipulate the contract logic (famously used in The DAO hack).¹⁵

Timestamp Dependency Contracts relying on `block.timestamp` (or `now`) can be misled by miners, who can influence timestamps within limits, affecting auctions, randomness, and time-locks.¹⁶

Transaction Ordering Dependence (TOD) When a contract’s outcome hinges on the order of pending transactions, attackers can front-run or back-run others in the mempool to gain unfair advantage.¹⁷

3.4 SmartBugs Schema

The SmartBugs analysis produces one record per contract execution and per tool, with the following fields:

- `filename` Relative path to the analyzed contract file.
- `basename` File name without directory components.
- `toolid` Identifier of the analysis tool (e.g., `oyente`, `oyente+`).
- `toolmode` Specific mode or configuration flag used during execution. (e.g., `runtime`, `solidity`)
- `parser_version` Version of the SmartBugs results parser employed.
- `runid` Unique ID for this particular analysis run. (Uses the following format: `YYYYMMDD_HHmm`)

¹⁵<https://hackernoon.com/hack-solidity-reentrancy-attack>

¹⁶<https://owasp.org/www-project-smart-contract-top-10/2023/en/src/SC03-timestamp-dependence.html>

¹⁷<https://scs.owasp.org/SCWE/SCSVS-ARCH/SCWE-052/>

- **start** Unix timestamp marking the start of analysis. (e.g., 1748651853.85692)
- **duration** Total execution time in seconds.
- **exit_code** Process exit status. Its meaning depends on the tool. For Oyente, 0 indicates an analysis without findings, and 1 one with findings. Numbers greater than 1 indicate an error.
- **findings** Array of detected vulnerability identifiers, separated by a comma.
- **infos** Array of informational messages emitted by the tool, separated by a comma.
- **errors** Array of error messages encountered during analysis, separated by a comma. Errors are exceptional conditions detected and reported by the tool itself.
- **fails** Array of failure messages or crash reports, separated by a comma. Failures are exceptional conditions detected only by the language interpreter or the operating system.

4 Run Summary

Table 4 summarizes the SmartBugs executions we performed to compare Oyente and Oyente+. Each run is identified by its timestamped RunID, execution mode, total contracts scanned¹⁸, and dataset.

RunID	Tool Mode	Total Rows	Dataset
20250530_2136	runtime	3758	CGT
20250601_1858	runtime	3758	CGT
20250605_0750	runtime	3758	CGT
20250604_0900	solidity	2921	CGT
20250604_1955	solidity	2924	CGT
20250605_1459	runtime	3598	Skelcodes
20250605_2329	runtime	3598	Skelcodes
20250611_1051	solidity	3598	Skelcodes
20250612_0648	solidity	3598	Skelcodes

Table 4: Summary of SmartBugs runs comparing Oyente and Oyente+.

We executed nine runs in total, covering both **runtime** and **solidity** modes across the CGT and Skelcodes datasets. The very first run (20250530_2136) served as a pre-fix baseline before applying a correction for the Callstack_Depth_Attack vulnerability. All subsequent runs were organized into paired executions to verify result consistency:

- 20250601_1858, 20250605_0750 (runtime on CGT)
- 20250604_0900, 20250604_1955 (solidity on CGT)
- 20250605_1459, 20250605_2329 (runtime on Skelcodes)
- 20250611_1051, 20250612_0648 (solidity on Skelcodes)

5 Results: Vulnerability Comparison

For each run and vulnerability, we compare detection by Oyente and Oyente+. Since the runs were done in pairs, as discussed in the previous section, we also took the result consistency into account.

¹⁸This is double the amount of the statistics presented in Table 1, because the values are counted per tool.

5.1 Callstack Depth Attack Vulnerability Analysis

Table 5 summarizes the detection of the *Callstack_Depth_Attack_Vulnerability* across our SmartBugs runs.

The very first execution (RunID 20250530_2136) revealed a defect in Oyente+, which failed to flag any instances (0/132) of this vulnerability. After applying a targeted bugfix¹⁹, subsequent paired runs in **runtime** mode on the CGT dataset (20250601_1858, 20250605_0750) show that Oyente+ now detects 103 of the 132 cases originally found by Oyente, with perfect run-to-run consistency. Similar improvements are observed in **solidity** mode on CGT (20250604_0900, 20250604_1955) and **runtime** mode on Skelcodes (20250605_1459, 20250605_2329), confirming the bugfix’s effectiveness.

Relevance of Callstack Depth Attacks Today

Although Oyente still achieves higher raw recall on this check, *call-depth attacks* have been rendered impractical by protocol-level changes. Since the Tangerine Whistle hard fork (October 2016), EIP-150’s²⁰ “63/64 rule” ensures that every contract call reserves at least 1/64 of the caller’s gas, preventing an attacker from exhausting the call stack before running out of gas. Contemporary best-practice guides therefore mark call-depth attacks as deprecated and of historical interest only.

Oyente+ Advantage On Skelcodes In Solidity Mode

In **solidity** mode on the Skelcodes dataset (RunIDs 20250611_1051, 20250612_0648), Oyente+ significantly outperforms Oyente (258/259 vs. 5 detections). This inversion stems from version support: vanilla Oyente is limited to Solidity versions up to 0.4.26 and EVM version up to 1.7.3, and emits warnings or fails on newer compiler/EVM versions. In contrast, Oyente+ fully supports recent Solidity (0.5.x–0.8.x) and EVM releases, allowing it to analyze the vast majority of contracts in Skelcodes without compatibility issues.

RunID	Contracts	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	132	132	0	0	0	1747
20250601_1858	1879	132	29	103	0	103	1747
20250605_0750	1879	132	29	103	0	103	1747
20250604_0900	1462	98	31	73	6	67	1358
20250604_1955	1462	98	31	75	8	67	1356
20250605_1459	1799	541	220	326	5	321	1253
20250605_2329	1799	541	220	326	5	321	1253
20250611_1051	1799	5	2	259	256	3	1538
20250612_0648	1799	5	2	258	255	3	1539

Table 5: Detection summary for *Callstack_Depth_Attack_Vulnerability*.

5.2 Integer Overflow Analysis

Table 6 summarizes the detection of the *Integer_Overflow* vulnerability across our SmartBugs runs. All **runtime**-mode executions returned zero detections, confirming that Oyente and Oyente+ can only identify overflow issues by inspecting the Solidity source.

In **solidity** mode on the CGT dataset (RunIDs 20250604_0900, 20250604_1955), Oyente flagged 1 100/1 088 overflow cases, respectively, while Oyente+ flagged 1 245 cases in both runs. The overlap of 1 088/1 079 contracts yields a recall of approximately 99 % for Oyente+ relative to Oyente, and shows 157/166 additional flags.

On Skelcodes in **solidity** mode (RunIDs 20250611_1051, 20250612_0648), vanilla Oyente detected only 14/13 overflows (0.8 % of contracts) due to its lack of support for Solidity $\geq 0.5.x$ and EVM $\geq 1.7.3$, whereas Oyente+ detected 1 281/1 281 cases (71 % of contracts), demonstrating its clear advantage on recent sources.

¹⁹https://github.com/smartbugs/oyente_plus/commit/19aa6a64d0b758489fe42dcc5fd721537bc2b843

²⁰<https://eips.ethereum.org/EIPS/eip-150>

RunID	Contracts	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	0	0	0	0	0	1879
20250601_1858	1879	0	0	0	0	0	1879
20250605_0750	1879	0	0	0	0	0	1879
20250604_0900	1462	1100	12	1245	157	1088	205
20250604_1955	1462	1088	9	1245	166	1079	208
20250605_1459	1799	0	0	0	0	0	1799
20250605_2329	1799	0	0	0	0	0	1799
20250611_1051	1799	14	0	1281	1267	14	518
20250612_0648	1799	13	0	1281	1268	13	518

Table 6: Detection summary for *Integer_Overflow*.

5.3 Integer Underflow Analysis

Table 7 summarizes the detection of the *Integer_Underflow* vulnerability. As with overflow, no underflows are detected in any `runtime`-mode runs, underscoring that underflow checks require source-level analysis.

In CGT `solidity` mode (20250604_0900, 20250604_1955), Oyente found 678/675 underflows, while Oyente+ found 1 077/1 078, with an overlap of 676/673 cases. This corresponds to a recall of roughly 99 % for Oyente+, with approx. 400 additional flags—again reflecting more aggressive detection of borderline patterns by the updated parser.

On Skelcodes (`solidity` mode, 20250611_1051, 20250612_0648), Oyente’s support gap limits it to 20/23 underflow flags (1.1 %), whereas Oyente+ identifies 1 262/1 263 cases (70 %). Apparently, Oyente+ copes differently with features of recent Solidity versions than Oyente, reinforcing that newer Solidity versions only become analyzable with the Oyente+ enhancements.

RunID	Contracts	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	0	0	0	0	0	1879
20250601_1858	1879	0	0	0	0	0	1879
20250605_0750	1879	0	0	0	0	0	1879
20250604_0900	1462	678	2	1077	401	676	383
20250604_1955	1462	675	2	1078	405	673	382
20250605_1459	1799	0	0	0	0	0	1799
20250605_2329	1799	0	0	0	0	0	1799
20250611_1051	1799	20	0	1262	1242	20	537
20250612_0648	1799	23	0	1263	1240	23	536

Table 7: Detection summary for *Integer_Underflow*.

5.4 Re-Entrancy Vulnerability Analysis

Table 8 summarizes the detection of the *Re_Entrancy_Vulnerability* across our SmartBugs runs.

Across both CGT and Skelcodes datasets, Oyente+ consistently reports more re-entrancy issues than vanilla Oyente. In the CGT runs (both `runtime` and `solidity` modes), Oyente+ finds up to 80 cases per run versus 40 by Oyente, with an overlap of about 40 contracts. On Skelcodes, the gap widens: in `runtime` mode, Oyente+ flags 110/112 versus only 1/2 by Oyente; in `solidity` mode, Oyente+ detects 86/89 cases while Oyente detects 0/1. This reflects Oyente’s lack of support for newer Solidity/EVM versions, which causes it to miss or abort on many recent contracts.

5.5 Timestamp Dependency Analysis

Table 9 shows the detection of the *Timestamp_Dependency* vulnerability.

RunID	Contracts	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	40	0	65	25	40	1814
20250601_1858	1879	39	1	78	40	38	1800
20250605_0750	1879	40	1	80	41	39	1798
20250604_0900	1462	38	3	70	35	35	1389
20250604_1955	1462	38	2	66	30	36	1394
20250605_1459	1799	1	1	112	112	0	1686
20250605_2329	1799	2	1	110	109	1	1688
20250611_1051	1799	0	0	89	89	0	1710
20250612_0648	1799	1	0	86	85	1	1713

Table 8: Detection summary for *Re_Entrancy_Vulnerability*.

RunID	Contracts	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	219	37	212	30	182	1630
20250601_1858	1879	210	47	261	98	163	1571
20250605_0750	1879	212	46	260	94	166	1573
20250604_0900	1462	193	45	225	77	148	1192
20250604_1955	1462	191	45	222	76	146	1195
20250605_1459	1799	4	1	46	43	3	1752
20250605_2329	1799	4	1	46	43	3	1752
20250611_1051	1799	1	0	41	40	1	1758
20250612_0648	1799	1	0	40	39	1	1759

Table 9: Detection summary for *Timestamp_Dependency*.

Both tools detect hundreds of timestamp-dependency issues in CGT across runs, with Oyente+ finding slightly more unique cases (up to 98 extra flags in initial CGT runs). In Skelcodes, detection drops sharply—vanilla Oyente flags only 1/4 cases per run, while Oyente+ still identifies 40/46 instances, again illustrating the impact of supporting recent Solidity/EVM versions.

5.6 Transaction Ordering Dependence Analysis

Table 10 summarizes the detection of the *Transaction_Ordering_Dependence_TOD* vulnerability across our SmartBugs runs. Across both datasets and modes, Oyente+ consistently detects more Transaction Ordering Dependence issues than vanilla Oyente.

RunID	Contracts	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	567	70	648	151	497	1161
20250601_1858	1879	561	53	760	252	508	1066
20250605_0750	1879	558	54	757	253	504	1068
20250604_0900	1462	436	53	534	151	383	875
20250604_1955	1462	435	53	527	145	382	882
20250605_1459	1799	7	1	166	160	6	1632
20250605_2329	1799	7	1	164	158	6	1634
20250611_1051	1799	2	0	115	113	2	1684
20250612_0648	1799	2	0	112	110	2	1687

Table 10: Detection summary for *Transaction_Ordering_Dependence_TOD*.

- CGT, runtime mode (20250530_2136, 20250601_1858): Oyente flags 561/567 cases; Oyente+ flags 648/760, with overlaps of 497/508, yielding net gains of +81 to +199 detections.

- **CGT, solidity mode** (20250604.0900, 20250604.1955): Oyente reports 435/436; Oyente+ reports 527/534, sharing 382/383 contracts and gaining +92 to +98 additional flags.
- **Skelcodes, runtime mode** (20250605.1459, 20250605.2329): Oyente finds only 7 cases; Oyente+ finds 164/166, with 6 overlaps, for a +157 to +159 increase.
- **Skelcodes, solidity mode** (20250611.1051, 20250612.0648): Oyente reports 2; Oyente+ reports 112/115, including the two contracts by Oyente and adding +110/+113 new ones.

This consistent increase demonstrates Oyente+’s enhanced capability to detect transaction ordering vulnerabilities, particularly in more recent contract sets where vanilla Oyente’s static analysis and version limitations hinder coverage.

6 Error And Failure Analysis

6.1 Error Analysis

Table 11 summarizes the number of contracts whose analysis aborted with an error in each SmartBugs run. Notably:

- All **runtime-mode** runs (20250530.2136, 20250601.1858, 20250605.0750, 20250605.1459, 20250605.2329) completed without errors for both tools.
- In **solidity-mode** on the CGT dataset (20250604.0900, 20250604.1955), both Oyente and Oyente+ failed to compile exactly 36 contracts.
- On the full Skelcodes solidity runs (20250611.1051, 20250612.0648), Oyente encountered 596 compile errors (with 325 unique failures), whereas Oyente+ reduced errors to 271 total (eliminating all unique failures). This represents a 54 % reduction in aborts due to enhanced version support.

Run ID	Total	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530.2136	1879	0	0	0	0	0	1879
20250601.1858	1879	0	0	0	0	0	1879
20250605.0750	1879	0	0	0	0	0	1879
20250604.0900	1462	36	0	36	0	36	1423
20250604.1955	1462	36	0	36	0	36	1426
20250605.1459	1799	0	0	0	0	0	1799
20250605.2329	1799	0	0	0	0	0	1799
20250611.1051	1799	596	325	271	0	271	1203
20250612.0648	1799	596	325	271	0	271	1203

Table 11: Summary of contracts with errors detected by Oyente and Oyente+.

6.1.1 Error Type Breakdown

Every error recorded was a Solidity compilation failure. Table 12 drills into the count of these failures by tool and run. These results confirm that:

- All compilation errors occur only in **solidity** mode.
- The identical counts for CGT solidity runs indicate that both tools share the same limitations on that dataset or contracts in this dataset include compilation errors.
- On Skelcodes, Oyente+’s expanded Solidity/EVM support cuts compilation failures by over half, greatly improving overall robustness.

Run ID	Error Type	Oyente	Oyente+
20250604_0900	Solidity compilation failed	36	36
20250604_1955	Solidity compilation failed	36	36
20250611_1051	Solidity compilation failed	596	271
20250612_0648	Solidity compilation failed	596	271

Table 12: Most frequent errors encountered during analysis.

6.2 Failure Analysis

This section examines contracts whose analysis terminated with a “fail” status rather than completing (either with findings or cleanly). Table 13 gives an overview per run, and Table 14 drills into the most common fail prefixes (first 25 characters of the error message) to approximate distinct failure modes.

Run ID	Total	Oyente	Only Oyente	Oyente+	Only Oyente+	Both	Neither
20250530_2136	1879	9	8	1	0	1	1870
20250601_1858	1879	8	7	3	2	1	1869
20250605_0750	1879	8	7	3	2	1	1869
20250604_0900	1462	85	70	31	16	15	1358
20250604_1955	1462	86	70	34	18	16	1358
20250605_1459	1799	16	16	21	21	0	1762
20250605_2329	1799	17	17	20	20	0	1762
20250611_1051	1799	1166	1037	144	15	129	618
20250612_0648	1799	1166	1033	148	15	133	618

Table 13: Contracts that ended in a “fail” status per run.

6.2.1 Failure Type Details

Table 14 breaks down the dominant failure modes by run, normalized to high-level exception or Docker exit codes. Several key observations emerge:

- **CGT Runtime Mode (20250530_2136, 20250601_1858, 20250605_0750):** Failures are rare (up to 4 per run) and dominated by `DOCKER_SEGV` and `ValueError` in Oyente. Oyente+ eliminates Docker segmentation faults entirely, instead showing occasional timeouts (`DOCKER_TIMEOUT`) and matching `NameError` counts, indicating improved low-level stability.
- **CGT Solidity Mode (20250604_0900, 20250604_1955):** Both tools equally share JSON decoding errors (18 total). Oyente most frequently aborts with `RuntimeError` (60 total failures each) and Docker segmentation faults, reflecting generic harness crashes. Oyente+ reduces `RuntimeError` from 60 to 1 but incurs more timeouts, suggesting parser robustness improvements at the cost of longer executions.
- **Skelcodes Runtime Test (20250605_1459, 20250605_2329):** Large contracts trigger Docker OOM kills (`DOCKER_KILL_OOM`, 15/16 in Oyente) and timeouts (19/21 in Oyente+). Oyente+ removes all OOM kills but exhibits timeouts, highlighting better memory handling but continued high resource demands.
- **Skelcodes Solidity Mode (20250611_1051, 20250612_0648):** Oyente aborts on 1 166 contracts (58%), chiefly via `RuntimeError` (765/754), `IndexError` (327/337), and `KeyError` (72). In contrast, Oyente+ cuts total failures to about 160, with only a handful of `RuntimeError` (6), `TypeError` (95), and timeouts (40/45), demonstrating robust version support while exposing new edge-case exceptions for further optimization.

Overall, Oyente+ reduces low-level crashes and OOM failures, shifting the residual failure profile toward higher-level parser or runtime exceptions that can be targeted in future development.

- CGT `runtime`-mode runs (20250530_2136, 20250601_1858, 20250605_0750) show only single-digit failures, mostly unique to one tool or the other.
- CGT `solidity`-mode (20250604_0900, 20250604_1955) exhibits about 85 failures in Oyente versus about 30 in Oyente+, indicating Oyente+'s improvements cut failures by over 60 %.
- Skelcodes solidity runs (20250611_1051, 20250612_0648) reveal large gaps: Oyente fails on 1 166 contracts (58 %), while Oyente+ reduces that to about 140 (8 %), a more than 85 % reduction in crashes and aborts.

Run ID	Error Type	Oyente	Oyente+	Total
20250530_2136	DOCKER_SEGV	4	0	4
	ValueError	4	0	4
	NameError	1	1	2
20250601_1858	ValueError	4	0	4
	DOCKER_SEGV	3	0	3
	DOCKER_TIMEOUT	0	2	2
	NameError	1	1	2
20250605_0750	ValueError	4	0	4
	DOCKER_SEGV	3	0	3
	DOCKER_TIMEOUT	0	2	2
	NameError	1	1	2
20250604_0900	RuntimeError	60	1	61
	json.decoder.JSONDecodeError	9	9	18
	DOCKER_TIMEOUT	0	13	13
	DOCKER_SEGV	8	0	8
	FileNotFoundError	3	3	6
	IndexError	3	2	5
	execution failed	1	1	2
	ValueError	0	2	2
	NameError	1	0	1
20250604_1955	RuntimeError	60	1	61
	json.decoder.JSONDecodeError	9	9	18
	DOCKER_TIMEOUT	0	16	16
	DOCKER_SEGV	9	0	9
	FileNotFoundError	3	3	6
	IndexError	3	2	5
	execution failed	1	1	2
	ValueError	0	2	2
	NameError	1	0	1
20250605_1459	DOCKER_TIMEOUT	0	21	21
	DOCKER_KILL_OOM	15	0	15
	ValueError	1	0	1
20250605_2329	DOCKER_TIMEOUT	0	19	19
	DOCKER_KILL_OOM	16	0	16
	symExec.TimeoutError	0	1	1
	ValueError	1	0	1
20250611_1051	RuntimeError	765	6	771
	IndexError	327	0	327
	TypeError	0	96	96
	KeyError	72	0	72
	DOCKER_TIMEOUT	0	40	40
	ValueError	0	2	2
	DOCKER_SEGV	1	0	1
	NameError	1	0	1
20250612_0648	RuntimeError	754	6	760
	IndexError	337	0	337
	TypeError	0	95	95
	KeyError	72	0	72
	DOCKER_TIMEOUT	0	45	45
	NameError	2	0	2
	ValueError	0	2	2
	DOCKER_SEGV	1	0	1

Table 14: Detailed failure types by run, normalized to exception or Docker exit codes.

7 Runtime Performance and Timeout Analysis

Using only successful runs (no errors, no failures), we computed per-tool latency percentiles and averages, and then counted overall failures and timeouts:

Tool	Median [s]	95th %ile [s]	Mean [s]	# Successes	Timeouts
Oyente	3.46	33.4	8.99	11 934	0
Oyente+	23.7	336.4	75.2	14 800	163

Table 15: Summary of successful-run durations and timeout counts.

Key observations:

- **Speed on supported contracts:** Oyente is substantially faster—its median successful run is only 3.5 s (95 % of runs under 33 s), compared to Oyente+’s 23.7 s median (95 % under 336 s).
- **Robustness:** Oyente+ completes more analyses overall (58 % success rate, 14 800 runs) than Oyente (52 % success, 11 934 runs), reflecting far fewer outright failures (416 vs. 2617).
- **Timeout behavior:** Oyente never hit the user-configured timeout, whereas Oyente+ timed out 163 times—an expected trade-off given its longer average runtimes on complex or newer contracts.

For the subset of contracts that both tools analyse to completion without raising an error, Oyente records shorter median runtimes. Yet Oyente also exhibits instances of silent termination that are not captured as explicit errors or timeouts, so we cannot assert that it is the faster tool overall. In contrast, Oyente+ achieves broader coverage and far fewer observable failures, albeit with longer measured runtimes for the contracts it completes and with a modest number of timeouts.

7.1 EVM Code Coverage Analysis

To measure how much of each contract’s bytecode was explored, we extracted the "EVM Code Coverage: xx.x%" entries from the `infos` array, used a regular expression to pull out the numeric percentage, cast it to `NUMERIC`, and then averaged those values per (`runid`, `toolid`).

Interpretation:

- Across all runs, *Oyente+* consistently achieves higher average coverage than *Oyente*—often by 15–20 percentage points on CGT and by over 60 points on Skelcodes runtime runs—thanks to its support for recent Solidity/EVM versions.
- Higher coverage correlates with longer analysis times: Oyente+’s richer exploration yields slower median runtimes (23.7 s vs. 3.5 s) and is responsible for its 163 timeouts, whereas Oyente never times out.
- The Skelcodes runtime-mode runs (20250605_1459, 20250605_2329) highlight this trade-off most starkly: Oyente covers only 8 % of bytecode before failing on compatibility, while Oyente+ covers 72 % but occasionally exceeds the timeout threshold.

RunID	Tool	Avg. Coverage [%]
20250530_2136	Oyente	63.85
	Oyente+	77.56
20250601_1858	Oyente	62.94
	Oyente+	81.78
20250605_0750	Oyente	63.02
	Oyente+	81.66
20250604_0900	Oyente	82.11
	Oyente+	88.35
20250604_1955	Oyente	81.88
	Oyente+	88.31
20250605_1459	Oyente	8.07
	Oyente+	71.99
20250605_2329	Oyente	8.17
	Oyente+	71.87
20250611_1051	Oyente	60.97
	Oyente+	82.32
20250612_0648	Oyente	61.97
	Oyente+	82.37

Table 16: Average EVM code coverage by run and tool.

8 Areas of Improvement in Oyente+

Although Oyente+ delivers substantially higher EVM code coverage and far fewer outright failures than vanilla Oyente, our quantitative analyses (Tables 15, 16, 13) highlight several key areas for further enhancement:

- **Fix new Exceptions in Oyente+:**

- *Observation:* With Oyente+ some new exceptions were introduced, that are not present in Oyente:
 - * `TypeError: 'NoneType' object is not subscriptable`
 - * `ValueError: non-hexadecimal number found in fromhex() arg at position X`
- *Recommendation:* Apart from general improvement of exception and error handling in Oyente+, focus on fixing these general issues to improve the stability of the tool further.

- **Exception and Error Handling:**

- *Observation:* Residual parser/runtime exceptions (e.g. `RuntimeError`, `TypeError`, JSON decode errors) still account for 416 failures (Table 13).
- *Recommendation:* Harden input preprocessing—sanitize and validate source maps and JSON artifacts before symbolic execution. Normalize exception flows by catching and categorizing low-level errors earlier, improving the tool’s resilience against malformed inputs.

- **Analysis Performance and Timeouts:**

- *Observation:* Oyente+’s median successful runtime (23.7s) is nearly $7\times$ slower than Oyente’s (3.46s), and its P95 tail approaches 6 min, leading to 163 timeouts under our current limits.
- *Recommendation:* Profile and optimize hotspot routines in symbolic execution and bytecode traversal (e.g. the AST walker and constraint solver) to reduce per-contract latency.

- **Expand and streamline unit testing**

- *Observation:* Oyente+ currently relies on a sparse, end-to-end test harness, which makes it cumbersome to isolate and verify individual components.
- *Recommendation:* Adopt a lightweight testing framework (e.g. `pytest`) and aim for at least 70 % coverage on core modules.
 - * Identify critical functions (AST parsing, constraint solving, source-map resolution) and write fine-grained tests that mock external dependencies.
 - * Use parameterized tests to cover edge cases (empty inputs, malformed bytecode, unusual pragma versions).
 - * Integrate coverage checks into CI so regressions are caught automatically on every pull request.

- **Refactor for modular architecture**

- *Observation:* The current codebase mixes parsing, symbolic execution, and I/O in tightly coupled scripts, making maintenance and feature additions error-prone.
- *Recommendation:* Decompose the code into well-defined packages and interfaces:
 - * Extract parsing logic, symbolic execution engine, and result-serialization into separate modules with clear public APIs.
 - * Apply dependency-inversion: higher-level analysis routines should depend on abstract interfaces rather than concrete implementations.
 - * Add documentation stubs and type annotations for each module to improve readability and enable static analysis.

Addressing these areas will help Oyente+ achieve both the robustness and the efficiency required for large-scale smart-contract auditing on current Solidity/EVM codebases.

9 Conclusion

In this work, we have presented Oyente+, a fork of the original Oyente vulnerability scanner updated to support recent EVM (v1.15.10) and Solidity (v0.5.x–v0.8.x) versions. By integrating Oyente+ into the SmartBugs framework and performing nine test runs with the CGT and Skelcodes datasets, we demonstrated that:

- **Extended version support** yields significantly higher vulnerability recall on recent contracts (e.g. +2500 detections of arithmetic and ordering bugs on Skelcodes) and reduces outright failures by over 85% (1166 to 144 in solidity mode).
- **Bugfixes** (e.g. call-stack detection) restored parity with vanilla Oyente on legacy checks, while obsoleting deprecated attack vectors (call-depth attacks are no longer feasible under EIP-150).
- **Performance trade-offs:** Oyente+ achieves 15–20% higher code coverage on CGT and 60% on Skelcodes, at the cost of a 7× slower median runtime and occasional timeouts (163 vs. 0).

Taken together, these results confirm that Oyente+ is both more robust and more comprehensive on recent Solidity/EVM codebases, at the expense of additional execution time. Future work should focus on:

- *Profiling and optimizing* the symbolic-execution hotspots to reduce median runtime and P95 tail.
- *Hardening exception flows* around parser and I/O edges to eliminate the remaining 416 failures and the handful of new TypeError/ValueError cases.
- *Modular refactoring* and expanded unit testing ($\geq 70\%$ coverage) to lock in correctness and accelerate further feature development.

Acknowledgments

The author would like to thank Gernot Salzer and Monika di Angelo for their ongoing support, incisive feedback, and for providing the CGT and Skelcodes datasets. He is also grateful to the SmartBugs team for welcoming Oyente+ into their framework and GitHub organization, and for their assistance during integration.

Generative AI Assistance

We leveraged the ChatGPT language model²¹ to refine the report’s grammar, spelling, and overall language clarity. All content was reviewed and edited by us to ensure factual accuracy and originality.

OpenAI’s ChatGPT was used during the development of code snippets and algorithm prototypes for Oyente+. It also aided in diagnosing and resolving implementation issues, helping to accelerate bug fixes and feature enhancements. All code was subsequently verified and tested.

Preliminary discussions and ideation were supported by queries to ChatGPT. We validated and expanded upon the suggestions. All final content decisions remain with us.

²¹<https://chatgpt.com/>

Appendix

Verbatim Listing of Setup and SQL Commands

SmartBugs Installation And Execution

The following steps were taken to generate our results.

```
# Setup SmartBugs
git clone https://github.com/smartbugs/smartbugs.git
cd smartbugs
./install/setup-venv.sh; source venv/bin/activate

# Run analysis
./smartbugs -t oyente,oyente+ -f <dataset-location>/**/*.sol \
--processes <X> --mem-limit <Y> --timeout <Z>

# Reformat raw output and generate PostgreSQL-ready CSV
./reparse results
./results2csv -p results > <path_to_results_file>
```

Database Setup

Here you can find the commands used to install and setup a PostgreSQL database on Ubuntu.

```
# Install PostgreSQL and create user & database
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
sudo -u postgres createuser <username> --createdb
sudo -u postgres createdb <database_name> --owner=<username>
```

```
# Define table schema based on smartbugs output
```

```
psql <database_name> <<EOF
CREATE TABLE smartbugs_results (
filename          TEXT,
basename          TEXT,
toolid            TEXT,
toolmode          TEXT,
parser_version    TEXT,
runid             TEXT,
start             DOUBLE PRECISION,
duration          REAL,
exit_code         INT,
findings          TEXT[],
infos             TEXT[],
errors            TEXT[],
fails             TEXT[]
);
EOF
```

```
# Import smartbugs results and cast timestamps
```

```
psql <database_name> <<EOF
\copy <table_name> (filename, basename, toolid, toolmode, parser_version, runid, start, duration, exit_code)
FROM '<path_to_results_file>' WITH CSV HEADER;
ALTER TABLE <table_name>
```

```
ALTER COLUMN start TYPE timestamp without time zone
USING to_timestamp(start)::timestamp without time zone;
EOF
```

Verbatim Listings Of Dataset Analyses

The contract dataset statistics were queried via `grep` and `wc` CLI commands:

```
# Files w/ pragma
grep -c "pragma solidity" <path_to_dataset>/**/*.sol | grep -v ":0" | wc -l

# Files w/o pragma
grep -c "pragma solidity" <path_to_dataset>/**/*.sol | grep ":0" | wc -l

# Multi-pragma files
grep -c "pragma solidity" <path_to_dataset>/**/*.sol | grep -v ":1" | grep -v ":0" | wc -l
```

Via `grep` and `wc` CLI commands we queried the Solidity version distribution:

```
# Repeat (or loop) for versions 0.4.x up to 0.8.x
grep "pragma solidity" <path_to_dataset>/**/*.sol | grep "0.4." | wc -l
[...]
```

To extract solc version data from the sample zip and info.csv we used the following commands:

```
# Extract a list of unique contract names inside the ZIP
unzip -Z1 skelcodes_selection.zip | xargs -n1 basename | sort -u > names.txt

# Only keep CSV lines whose first field matches the list
awk -F',' 'NR==FNR {have[$1]; next} ($1 in have)' names.txt info.csv > info_filtered.csv
```

Verbatim Listing Of SQL Statements

This section shows the SQL statements used to query the results for our Tables from the database.

- SQL query to generate Table 4 - Run Summary:

```
SELECT DISTINCT runid, toolmode,
COUNT(*) OVER (PARTITION BY runid) AS rows_by_runid
FROM smartbugs_results
ORDER BY runid;
```

- SQL queries to generate Table 5, Table 6, Table 7, Table 8, Table 9, Table 10. Just replace `$vulnerability$` with the respective string representation:

```
WITH per_contract AS (
SELECT
  filename, runid,
  BOOL_OR(toolid = 'oyente' AND '$vulnerability$' = ANY(findings)) AS oyente_found,
  BOOL_OR(toolid = 'oyente+' AND '$vulnerability$' = ANY(findings)) AS oyente_plus_found
FROM smartbugs_results
GROUP BY runid, filename
```

```

)
SELECT
  runid,
  COUNT(*) AS total_contracts,
  SUM(CASE WHEN oyente_found THEN 1 ELSE 0 END) AS total_oyente,
  SUM(CASE WHEN oyente_found AND NOT oyente_plus_found THEN 1 ELSE 0 END) AS only_oyente,
  SUM(CASE WHEN oyente_plus_found THEN 1 ELSE 0 END) AS total_oyente_plus,
  SUM(CASE WHEN NOT oyente_found AND oyente_plus_found THEN 1 ELSE 0 END) AS only_oyente_plus,
  SUM(CASE WHEN oyente_found AND oyente_plus_found THEN 1 ELSE 0 END) AS total_both,
  SUM(CASE WHEN NOT oyente_found AND NOT oyente_plus_found THEN 1 ELSE 0 END) AS total_neither
FROM per_contract
GROUP BY runid
ORDER BY runid;

```

- SQL query to generate Table 11 - Error Summary:

```

WITH per_contract AS (
  SELECT
    filename,
    runid,
    BOOL_OR(toolid = 'oyente' AND array_length(errors, 1) > 0) AS oyente_found,
    BOOL_OR(toolid = 'oyente+' AND array_length(errors, 1) > 0) AS oyente_plus_found
  FROM smartbugs_results
  GROUP BY runid, filename
)
SELECT
  runid,
  COUNT(*) AS total_contracts,
  SUM(CASE WHEN oyente_found THEN 1 ELSE 0 END) AS total_oyente,
  SUM(CASE WHEN oyente_found AND NOT oyente_plus_found THEN 1 ELSE 0 END) AS only_oyente,
  SUM(CASE WHEN oyente_plus_found THEN 1 ELSE 0 END) AS total_oyente_plus,
  SUM(CASE WHEN NOT oyente_found AND oyente_plus_found THEN 1 ELSE 0 END) AS only_oyente_plus,
  SUM(CASE WHEN oyente_found AND oyente_plus_found THEN 1 ELSE 0 END) AS total_both,
  SUM(CASE WHEN NOT oyente_found AND NOT oyente_plus_found THEN 1 ELSE 0 END) AS total_neither
FROM per_contract
GROUP BY runid
ORDER BY runid;

```

- SQL query to generate Table 12 - Error Types:

```

SELECT
  runid,
  error,
  COUNT(*) AS total_contracts,
  SUM(CASE WHEN toolid = 'oyente' THEN 1 ELSE 0 END) AS total_oyente,
  SUM(CASE WHEN (toolid = 'oyente' AND NOT toolid != 'oyente+') THEN 1 ELSE 0 END) AS only_oyente,
  SUM(CASE WHEN toolid = 'oyente+' THEN 1 ELSE 0 END) AS total_oyente_plus,
FROM (
  SELECT runid, toolid, unnest(errors) AS error
  FROM smartbugs_results
) t
GROUP BY runid, error

```

```
ORDER BY runid, error;
```

- SQL query to generate Table 13 - Failure Summary:

```
WITH per_contract AS (
  SELECT
    filename,
    runid,
    BOOL_OR(toolid = 'oyente' AND array_length(fails, 1) > 0) AS oyente_found,
    BOOL_OR(toolid = 'oyente+' AND array_length(fails, 1) > 0) AS oyente_plus_found
  FROM smartbugs_results
  GROUP BY runid, filename
)
SELECT
  runid,
  COUNT(*) AS total_contracts,
  SUM(CASE WHEN oyente_found THEN 1 ELSE 0 END) AS total_oyente,
  SUM(CASE WHEN oyente_found AND NOT oyente_plus_found THEN 1 ELSE 0 END) AS only_oyente,
  SUM(CASE WHEN oyente_plus_found THEN 1 ELSE 0 END) AS total_oyente_plus,
  SUM(CASE WHEN NOT oyente_found AND oyente_plus_found THEN 1 ELSE 0 END) AS only_oyente_plus,
  SUM(CASE WHEN oyente_found AND oyente_plus_found THEN 1 ELSE 0 END) AS total_both,
  SUM(CASE WHEN NOT oyente_found AND NOT oyente_plus_found THEN 1 ELSE 0 END) AS total_neither
FROM per_contract
GROUP BY runid
ORDER BY runid;
```

- SQL query to generate Table 14 - Failure Types Detailed:

```
WITH truncated_fails AS (
  SELECT
    runid,
    toolid,
    LEFT(fail, 25) AS fail_prefix
  FROM smartbugs_results
  CROSS JOIN LATERAL unnest(fails) AS fail
)
SELECT
  runid,
  fail_prefix,
  SUM(CASE WHEN toolid = 'oyente' THEN 1 ELSE 0 END) AS count_oyente,
  SUM(CASE WHEN toolid = 'oyente+' THEN 1 ELSE 0 END) AS count_oyente_plus
FROM truncated_fails
GROUP BY runid, fail_prefix
ORDER BY runid, fail_prefix;
```

- SQL query to generate Table 15 - Runtime Summary:

```
WITH successes AS (
  SELECT
    runid, toolid, duration, fails, errors
  FROM smartbugs_results
```

```

WHERE cardinality(fails) = 0 AND cardinality(errors) = 0
)
SELECT
  toolid,
  PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY duration) AS median_secs,
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY duration) AS p95_secs,
  AVG(duration) AS mean_secs,
  COUNT(*) AS runs
FROM successes
GROUP BY toolid;

```

- SQL query to generate Table 16 - Coverage Summary:

```

WITH coverage_values AS (
  SELECT
    runid,
    toolid,
    -- extract the number between \: " and \%
    (regexp_replace(info, '^EVM Code Coverage: ([0-9.]+)%$', '\1')::NUMERIC) AS coverage_pct
  FROM smartbugs_results
  CROSS JOIN LATERAL unnest(infos) AS info
  WHERE info LIKE 'EVM Code Coverage:%'
)
SELECT
  runid,
  toolid,
  ROUND(AVG(coverage_pct)::NUMERIC, 2) AS avg_coverage_pct
FROM coverage_values
GROUP BY runid, toolid
ORDER BY runid, toolid;

```